

Relatório aula 007 - Paradigmas

Slide 2



Objetivo e ligação com o mundo real

Objetivo

O objetivo desta aula é entender **como as linguagens avaliam expressões e controlam o fluxo de execução**, para que o seu código faça exatamente o que parece fazer, **sem surpresas** de precedência, conversão ou desvio.

Quando o fluxo sai do controle

2014 - "goto fail" (Apple)

Uma linha `goto fail`; duplicada para a verificação de assinatura. iOS e macOS aceitavam certificados falsos.

2008 - Tocadores Zune (Microsoft)

No último dia de um ano bissexto, um laço `while` nunca terminava, e todos os aparelhos travaram.

PHP - comparação com ==

"0e4..." == "0e3..." é verdadeiro: logins que comparavam hashes com == aceitavam senhas erradas.

Para o engenheiro de software, isso significa:

- ler código alheio sem ser enganado pela indentação ou pela precedência
- evitar conversões implícitas perigosas (== do JS e do PHP)
- escolher a estrutura de controle mais legível para cada caso

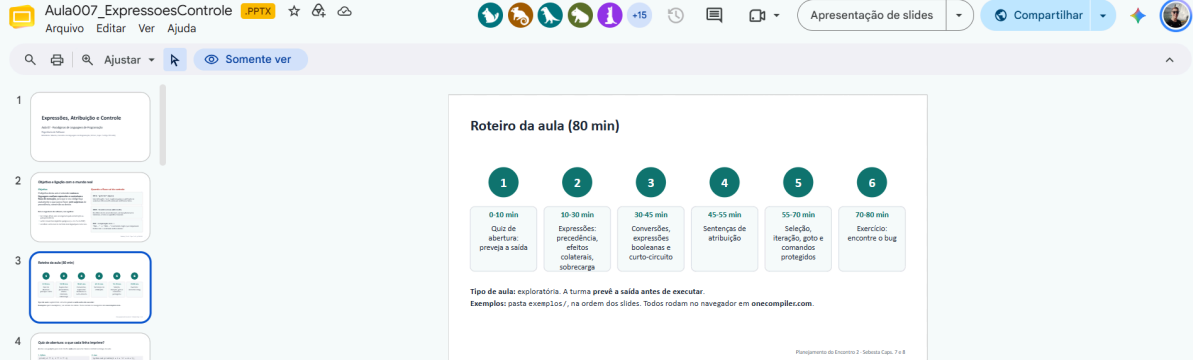
chaves obrigatórias.

- Zune (31/12/2008): os tocadores Zune de 30 GB da Microsoft travaram no mundo todo. O código que convertia "dias desde 1980" em ano entrava num laço `while` que não fazia progresso no dia 366 de um ano bissexto. No dia seguinte, voltaram a funcionar sozinhos. O exemplo está em `08_iteracao/zune.c`.
- PHP e "type juggling": ao comparar dois textos com `==`, o PHP os converte para número se ambos "parecem" números. "0e462..." e "0e830..." viram `0 x 10^N`, ou seja, `0 == 0`. Hashes MD5 que começam com "0e" seguidos só de dígitos ("magic hashes") fizeram sistemas de login aceitarem senhas erradas. O exemplo está em `04_conversoes/coercao.php`.

Mensagem: todos os três são erros de ESTRUCTURA DE CONTROLE ou de AVALIAÇÃO DE EXPRESSÕES, e não de "lógica de negócio".

Aqui é explicado o objetivo da aula, bem como foram mostrados exemplos que demonstram a importância de compreender o assunto da aula: Expressões, Atribuição e Controle.

Slide 3



Roteiro da aula (80 min)

1	2	3	4	5	6
0-10 min	10-30 min	30-45 min	45-55 min	55-70 min	70-80 min
Quiz de abertura, preveja a saída	Expressões: precedência, efeitos colaterais, sobrecarga	Conversões, expressões booleanas e curto-circuito	Sentenças de atribuição	Seleção, iteração, goto e comandos protegidos	Exercício: encontre o bug

Mostre o roteiro rapidamente (1 min).

Dinâmica de toda a aula: 1) mostrar o código; 2) a turma prevê a saída; 3) executar no OneCompiler; 4) explicar a regra da linguagem com o livro. O exercício final ("encontre o bug") pode valer como atividade avaliativa em sala.

Ficam para autopesquisa (último slide): expressões em Ruby e Lisp, expressões condicionais, implementação de seleção múltipla (tabelas de desvio), laços em linguagens funcionais e os iteradores de Ruby.

Slide com o roteiro previsto para a aula

Slide 4

Aula007_ExpressoesControle .PPTX

Arquivo Editar Ver Ajuda

Apresentação de slides

Compartilhar

Somente ver

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 - Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 - Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 - JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 - C

```
printf("%d\n", 3 > 2 > 1);
```

5 - Python

```
x = 0
print(x or "padrão", 3 > 2 > 1)
```

6 - JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

As respostas aparecem ao longo da aula - Sebesta Cap. 7

1. Python

Palpite: 4 64

Resultado real:

```
main.py x +
```

```
1 print(-2 ** 2, 2 ** 3 ** 2)
```

Console I/O AI Agent 139 ms

-4 512

2. Java

Palpite: Não irá compilar com erro de tipo

Resultado real:

```
Main.java x +
```

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         System.out.println(1+2+"3"+4+5);
6     }
7 }
```

Console I/O AI Agent 936 ms

3345

3. JavaScript

Palpite: 3 52

Resultado real:

JS index.js	+	Console	I/O	AI Agent	158 ms	✓
1 console.log("5" - 2, "5" + 2);		3 52				

4. C

Palpite: 3

Resultado real:

C Main.c	+	Console	I/O	AI Agent	170 ms	✓
1 #include <stdio.h> 2 int main() 3 { 4 printf("%d\n", 3 > 2 > 1); 5 }		0				

5. Python

Palpite: padrão True

Resultado real:

main.py	+	Console	I/O	AI Agent	136 ms	✓
1 x = 0 2 print(x or "padrão", 3 > 2 > 1)		padrão True				

6. JavaScript

Palpite: true false true

Resultado real:

JS index.js	+	Console	I/O	AI Agent	155 ms	✓
1 console.log(0 == "", "" == "0", 0 == "0");		true false true				

Slide 5

Expressões aritméticas: questões de projeto

- Quais são as regras de precedência?
- Quais são as regras de associatividade?
- Em que ordem os operandos são avaliados?
- Há restrições a efeitos colaterais?
- O usuário pode selecionar operadores?
- Que mistura de tipos é permitida?

Lisp dispensa a precedência (p. 306)
 $(+ (* 3 (* 4 5)))$: notação prefixa, e os parênteses dão a ordem.

de esquerda para a direita
 $(3 + 4) * 5 = 35$

de direita para a esquerda
 $3 + (4 * 5) = 23$

com precedência (*) antes de +
 $3 + 4 * 5 = 23$

APL, sempre de direita para a esquerda (p. 304)
 $3 \times 4 + 5 = 27$

Tempo: ~3 min.
 Sebesta (p. 301): as expressões aritméticas são formadas por operadores, operandos, parênteses e chamadas de função. Os operadores podem ser unários, binários ou ternários. Avaliar uma expressão exige duas ações: obter os operandos e executar as operações. As seis questões de projeto estão na lista à esquerda (p. 301). Elas organizam toda a primeira metade da aula. Exemplo do livro (p. 301): $a + b * c$ com $a=3$, $b=4$, $c=5$. Da esquerda para a direita dá 35, da direita para a esquerda dá 23. As regras de PRECEDÊNCIA (herdadas da matemática) decidem que * vem antes: 23. APL (p. 304) tem um só nível de precedência e avalia sempre da direita para a esquerda: $A \times B + C$ com 3, 4 e 5 dá $3 \times (4 + 5) = 27$. Ou seja, a precedência é uma convenção de projeto, e não uma lei. Lisp (p. 306) não precisa de precedência: tudo é prefixo e parentetizado, como em $(+ (* b c))$. Ver 11_autopesquisa/expressoes.lisp.

Nesse slide, foram apresentadas algumas a serem feitas a nível de projeto na hora de escolher a linguagem utilizada. Além disso foram mostrados exemplos de como a forma de tratar a precedência em operações pode afetar o resultado, reforçando a importância da decisão em questão

Slide 6

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
$-2 ** 2$	-4	erro de sintaxe (não há **)	
$2 ** 3 ** 2$	512	512	--
$1 + 2 + "3"$	TypeError	"33"	"33"
$-7 / 2$ (inteiro)	-4 (com //)	-3.5	-3
-7×2	1	-1	-1

Associatividade (p. 303)
 Esquerda: $* / \% + -$
 Direita: $**$ (Python, JS, Ruby, Fortran), $++$, $--$, menos unário
 Visual Basic: à esquerda!

Excel
 -2^2 dá 4: lá o menos unário vence o ^.

Na dúvida, use parênteses (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Respostas do quiz: $-2 ** 2 = -4$ em Python (o ** tem precedência maior que o menos unário). Em JavaScript isso é ERRO DE SINTAXE: os projetistas preferiram obrigar os parênteses a deixar a expressão ambígua. $2 ** 3 ** 2 = 2 ** 9 = 512$. Curiosidade do mundo real: no Excel, $=-2^2$ dá 4, porque o menos unário tem precedência MAIOR que o ^, ao contrário da matemática e do Python. $1 + 2 + "3" + 4 + 5$: em Java dá "3345" (esquerda para a direita, e o + muda de significado no meio). Em Python é TypeError (int + str não é permitido). Divisão e resto: Java e C truncam em direção a zero ($-7/2 = -3$, $-7\%2 = -1$). Python arredonda para baixo ($-7/2 = -4$, $-7\%2 = 1$). O livro observa (nota da p. 303) que antes do C99 isso dependia da implementação. Parênteses (p. 305): sempre podem mudar a ordem. Um projeto sem precedência, só com parênteses, seria mais simples, mas tedioso de ler e escrever.

Aqui, são mostradas algumas operações e seus respectivos resultados em Python, JS e Java, demonstrando a precedência e associatividade. Por exemplo, no caso da associatividade que é esquerda para $* / \% + -$ direita para $**$ (Python, JS, Ruby, Fortran), $++$, $--$, menos unário. Esse slide também parcialmente explica alguns comportamentos do primeiro quiz.

Slide 7

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;
int fun1(void) {
    a = 37; /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8 Java
a ordem é garantida: da esquerda para a direita (p. 308)

20 C (gcc 14)
a ordem não é definida pelo padrão

20 Go
a especificação não define a ordem de leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem de avaliação de $f(x) + f(x)$ pode virar $2 * f(x)$.

Resultados REAIS no OneCompiler (testados): Java = 8, C (gcc 14) = 20, Go = 20.

- Java (p. 308): a definição da linguagem GARANTE a avaliação da esquerda para a direita.
- C: o padrão não define a ordem, e cada compilador pode fazer diferente (inclusive o mesmo compilador com outra otimização).
- Go: a especificação define a ordem das CHAMADAS de função, mas não a leitura das variáveis em relação a elas.

Duas soluções possíveis (p. 308): proibir efeitos colaterais funcionais (difícil em linguagens imperativas) ou fixar a ordem (atrapalha otimizações). Não há solução perfeita.

Transparência referencial (p. 308-309): se duas expressões com o mesmo valor podem ser trocadas uma pela outra sem mudar o programa. Funções puras garantem isso, e as linguagens funcionais puras são transparentes referencialmente (Cap. 15).

Pergunte: "qual das três escolhas é a melhor?" e discuta desempenho x previsibilidade.

Esse slide discute ordem dos operandos e efeitos colaterais. Aqui é mostrado um exemplo com uma expressão que soma uma variável declarada previamente com o resultado de uma função que a sobrescreve. O resultado é demonstrado em Java, C e Go, onde Java resulta em 8 por ser da esquerda para direita sempre, C não há ordem definida (é definido pelo compilador, e em Go que define a ordem das chamadas de função, mas não a leitura das variáveis em relação a elas. Não há uma solução perfeita, sempre havendo trade-offs.

Slide 8

Operadores sobrecarregados

- O mesmo símbolo com vários significados: + soma inteiros, reais e concatena (java)
- Risco (p. 309): 8 em C++ é 8 bit a bit e "endereço de"
- Definida pelo usuário (p. 404): C++, C#, Python, Rust, Kotlin. Java não permite

Python - sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):
        return Vetor(self.x + outro.x, self.y + outro.y)

    def __mul__(self, k):
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)  # Vetor(4, 6)
print(v * 3)                 # Vetor(12, 18)
```

Java

```
System.out.println("a" + 1); // 98
System.out.println("a" + 1); // a1
x.add(y).multiply(tres); // BigDecimal: sen +
```

que o compilador aceita.

Sobrecarga definida pelo usuário (9.11, p. 404): C++, C#, Python, Ruby, Kotlin e Rust permitem, cada uma com um mecanismo próprio. Python usa métodos especiais (`__add__`). Rust usa traits (`impl Add`). Java decidiu NÃO permitir, e por isso `BigDecimal` usa `a.add(b)` em vez de `a + b`.

Resposta do quiz: 'a' + 1 em Java dá 98, porque o char sofre coerção para int. Já "" + 'a' + 1 dá "a1".

Discussão: sobrecarga melhora a legibilidade (v1 + v2 para vetores ou matrizes) ou esconde custo e comportamento? Qual é a opinião da turma?

Esse slide trata da sobrecarga de operadores, ou seja, quando um operador faz mais de uma só coisa. A exemplo do operador "+" que em Java soma inteiros, floats e concatena strings. Também cita o perigo disso, bem como diferentes linguagens tratam sobrecargas definidas pelo usuário (no caso de Java, NÃO permite), com um exemplo prático em Python.

Slide 9

Conversões de tipos: alargamento, estreitamento e coerção

Java - coerção e cast

```
int grande = 123456789;
float f = grande; // alargamento implícito
System.out.println((int) f); // 123456792

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 288); // -56

byte b = 30, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)
System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 → int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(1)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a` com `a` digitado por engano no lugar de `c`, o Java converte e compila.

Tempo: ~4 min. DEMONSTRAÇÃO: Conversoes.java, depois conversoes.go e conversoes.rs.

Definições (p. 310-311):

- Estreitamento: converte para um tipo que não comporta todos os valores do original (double para int). Pode perder informação.
- Alargamento: converte para um tipo "maior" (int para double). Normalmente é seguro, MAS pode perder precisão. O exemplo do livro (p. 311): int de 32 bits tem cerca de 9 dígitos, e float de 32 bits tem cerca de 7. No OneCompiler, 123456789 vira 123456792.
- Coerção: conversão IMPLÍCITA feita pelo compilador. Cast: conversão EXPLÍCITA pedida pelo programador.
- Expressão de modo misto (p. 311): operandos de tipos diferentes. Quem permite precisa definir as coerções.

O livro (p. 312) mostra o custo: em Java, `d = b * a` com `a` sendo int "por engano" compila, porque a coerção esconde o erro de digitação. Por isso ML e F# NÃO permitem modo misto. Go e Rust seguem a mesma linha: nem int32 para int64 é automático.

Aqui são explicadas as conversões de tipos.

Estreitamento é quando o tipo que foi alterado não comporta todos os valores do original, podendo perder informação.

Alargamento é quando vai para um tipo que comporta mais informação, normalmente seguro mas pode perder precisão.

Coerção é a troca de tipo, podendo ser implícita ou explícita (casting).

O slide também exemplifica esse comportamento entre diferentes linguagens

Slide 10

Coerção implícita: JavaScript e PHP

JavaScript

```
// 3
"5" - 2 // 3
"5" + 2 // "52"
"5" * 2 // 10
true + 1 // 2
[] + [] // ""
null + 1 // 1
undefined + 1 // NaN
"5" + "a" + "a" + "a" // "5aNaNa"
```

PHP - "type juggling"

```
var_dump("0e123" == "0e456"); // true
var_dump("0e123" === "0e456"); // false
var_dump("1" == "00"); // true
md5("140610708") // "0e402097431006...."
md5("QNKCDZO") // "0e830400451993...."
// == entre os dois hashes: true
```

Regra prática
Em JavaScript e PHP, compare com `===` e `!==`, que não fazem coerção.

Tempo: ~4 min. DEMONSTRAÇÃO: coercao.js no console do navegador (F12) e coercao.php no onecompiler.com/php.

Respostas do quiz: "5" - 2 = 3, porque o - só existe para números. "5" + 2 = "52", porque o + concatena se um dos lados é texto. 0 == "" é true, "" == "0" é false e 0 == "0" é true. Ou seja, o == do JavaScript NÃO é transitivo.

"b" + "a" + "a" + "a" = "baNaNa"; o "+" é o mais unário aplicado a "a", que dá NaN.

Regra prática em JavaScript e PHP: use `===` e `!==` (não fazem coerção). Linters como o ESLint (regra eqeqeq) obrigam isso.

PHP (type juggling): "0e123" == "0e456" é true porque os dois textos parecem números em notação científica e valem 0. Os hashes MD5 de "240610708" e "QNKCDZO" começam com "0e" seguidos só de dígitos. Um login que fizesse md5(\$senha) == \$hash_salvo aceitaria uma senha pela outra. O PHP 8 corrigiu parte disso: "abc" == 0 passou a ser false, mas "0e..." == "0e..." continua true.

Ligação com o Sebesta: o custo da coerção é a perda de detecção de erros (p. 312). Linguagens de script escolheram facilidade de escrita em vez de confiabilidade.

Aqui são mostrados alguns exemplos e peculiaridades do JS e do PHP em relação a tipagem e coerção implícita

Slide 11

Expressões relacionais e booleanas

C - booleano é inteiro (p. 315)

```
int a = 8, b = 2, c = 1;
printf("a > b > c:"); // 0
// (a > b) vale 1, e 1 > 1 é falso
printf("a > b && (b > c):"); // 1
if (-1) printf("verdadeiro\n"); // 1
```

Python - comparações encadeadas

```
a, b, c = 8, 2, 1
print(a > b > c) # True: (a > b) and (b > c)
# contam como FALSO em um if:
# 0, [], {} e None
# contam como VERDADEIRO:
# "e" [e]
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleanos: 1 if com o inteiro não compila

tempo: ~3 min. DEMONSTRAÇÃO: relacionais.c e relacionais.py.

Sebesta (p. 313-314): os operadores relacionais têm precedência MENOR que os aritméticos ($a + 1 > 2 * b$ faz a aritmética primeiro). O resultado é booleano, exceto em C89, que não tinha tipo booleano: 0 é falso e qualquer outro valor é verdadeiro (p. 315). Resposta do quiz: em C, $3 > 2 > 1$ vale 0. O livro chama isso de "resultado incomum" (p. 315): $(3 > 2)$ dá 1, e $1 > 1$ é falso. A comparação entre b e c nunca acontece.

Em Python, a mesma expressão é ENCADEADA: $3 > 2 > 1$ significa $(3 > 2)$ and $(2 > 1)$, que é True, como na matemática.

"Truthiness": Python e JavaScript aceitam qualquer valor numa condição. 0, "", [], {} e None contam como falso, mas "0", [0] e "" contam como verdadeiro. Java, C#, Go e Rust EXIGEM um booleano no if, e o livro (p. 315) defende isso pela legibilidade e pela detecção de erros.

Perl e Ruby têm && e 'and' com precedências diferentes (p. 315).

Aqui é explicado como as linguagens tratam expressões relacionais e booleanas, como por exemplo 0 e vazio serem False em Javascript, o que não funciona em Java que aceita somente booleanos de fato

Slide 12

Avaliação em curto-circuito

Java - && faz curto-circuito, e não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;
// não avança lista.length
// não avança lista.length
```

JavaScript - ? e ??

```
const u = { nome: "Mig", endereco: null };
u.endereco.cidade // undefined, sem erro
const qtd = 0;
qtd || 10 // 10 (o 0 se perdeu!)
qtd ?? 10 // 0 (o 0 não se perdeu!)
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleanos: 1 if com o inteiro não compila

Cuidado com efeitos colaterais (p. 316)

Em $(a > b) || ((b++ / 3) / 3)$, o $b++$ só acontece quando $a <= b$. Em Python, b or "padrão" devolve "padrão": and e or devolvem um dos operandos.

Tempo: ~4 min. DEMONSTRAÇÃO: CurtoCircuito.java e depois curto_circuito.js.

Definição (p. 315): o resultado é determinado sem avaliar todos os operandos. Em $(a >= 0) \&\& (b < 10)$, se $a < 0$ o segundo termo é irrelevante. Exemplo do livro (p. 316): a busca while ((index < listlen) && (list[index] != key)). SEM curto-circuito, quando a chave não está na lista, list[listlen] seria acessado e lançaria uma exceção de índice. O curto-circuito protege o acesso.

Linguagens baseadas em C: && e || fazem curto-circuito. & e | com booleanos NÃO fazem (p. 317). Ruby, Perl, ML, F# e Python fazem curto-circuito em todos os operadores lógicos (p. 317).

Risco (p. 316): curto-circuito combinado com efeito colateral. Em $(a > b) || ((b++ / 3) / 3)$, o $b++$ só acontece às vezes.

Modernos: o ?. (encadeamento opcional) e o ?? (coalescência nula) de JavaScript, TypeScript, C# e Kotlin são curto-circuitos especializados em null.

O slide explica o curto circuito que é interromper a execução sem avaliar todos os operandos. Em java por exemplo && faz curto circuito, enquanto & não. Em Javascript temos ?. e ?? por exemplo.

Slide 13

Sentenças de atribuição

C - a atribuição é uma expressão (p. 319-320)

```
sum = count = 0; // múltiplos alvos
a = b + (c = d / b) - 1; // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... } // compila! atribui e testa y
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: if (x = y) compila
Java, C#	sim, mas o if exige booleano
Python	não, exceto o operador morsa :=
Go	não: nem x++ é expressão
Rust	vale (); if x = 5 não compila

Tempo: ~5 min. DEMONSTRAÇÃO: atribicao.c, atribicao.py e atribicao.go.

Formas (p. 317-321):

- Simples: quase todas as linguagens usam =. ALGOL usou := para não confundir com a igualdade.
- Composta: a += b (vem do ALGOL 68 e foi adotada por C).
- Unária: count++ e ++count. Em count++, o incremento acontece antes da negação (p. 319).
- Como EXPRESSÃO (p. 319-320): em C, JavaScript e Perl, a atribuição devolve o valor atribuído. Isso permite while ((ch = getchar()) != EOF) e sum = count = 0. O exemplo do livro a = b + (c = d / b) - 1 aparece no arquivo.
- O custo (p. 320): if (x = y) em vez de if (x == y) compila em C, Java e C# evitam o problema porque o if só aceita booleano.
- Múltipla (p. 320-321): Perl, Ruby, Lua, Python e Go permitem a, b = b, a (troca sem variável temporária).
- Funcional (p. 321): em ML e F#, val e let dão NOME a um valor, que nunca muda. O let de Rust é imutável por padrão, e "let x = x + 1" cria uma

Aqui são explicadas sentenças de atribuição, ou seja, como declaramos ou atribuímos variáveis.

Simples: a = 1

Composta: a += b

Unária: a++

Como expressão: a = b + (c = d / b) - 1

Múltipla: a, b = b, a

Funcional: Dão nome a um valor que não muda

Slide 14

Atribuição de modo misto

Java - só alargamento (p. 322)

```
double d = 3; // ok: alargamento
// int i = 3.7; // erro: estreitamento
int i = (int) 3.7; // 3 (cast explícito)
long l = i; // ok

i += 3.7; // COMPILA: cast escondido
System.out.println(i); // 6
```

C - coerção em qualquer direção

```
int i = 3.7; // 3, em silêncio
char c = 300; // 44 (só um aviso)
unsigned int u = -1; // 4294967295
```

Armadilha clássica

```
double taxa = acertos / total * 100; com inteiros dá 0.0: a divisão inteira acontece antes do alargamento para double.
```

Tempo: ~3 min. DEMONSTRAÇÃO: ModoMisto.java e modo_misto.c.

Questão de projeto (p. 321): a expressão precisa ter o mesmo tipo da variável que recebe o valor, ou vale coerção?

- C, C++ e Perl: coerção em qualquer direção. int i = 3.7 vira 3 em silêncio, e char c = 300 vira 44 (o gcc só avisa).
- Java e C# (p. 322): só permitem a coerção de ALARGAMENTO (int para float, mas não o contrário). Segundo o livro, proibir metade das atribuições de modo misto é um jeito simples e eficaz de aumentar a confiabilidade

Aqui o slide mostra como Java e C se comportam ao atribuir a variáveis de tipo x recebendo valores de um tipo diferente.

Slide 15

Estruturas de controle: quantas são necessárias?

Sequência
uma sentença após a outra

Seleção
escolher um caminho

Iteração
repetir enquanto uma condição vale

```
a = 1;  
b = a + 2;
```

```
if (x > 0) ...  
else ...
```

```
while (x > 0)  
x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (for, switch, do, foreach)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Sebesta, LIT ed. - §5.1 - p. 328-330

Tempo: ~2 min. Transição do Cap. 7 para o Cap. 8.
Sebesta (p. 328): as computações imperativas avaliam expressões e atribuem valores, mas programas úteis precisam de mais dois mecanismos: SELECIONAR entre caminhos e REPETIR sentenças.
Böhm e Jacopini (1966) provaram que qualquer algoritmo representável em fluxograma pode ser escrito só com sequência, seleção de dois caminhos e laço com pré-teste. O goto é desnecessário. Esse resultado apoiou a programação estruturada.
Por que as linguagens têm tantas estruturas a mais (for, switch, do-while, foreach...)? Pela facilidade de escrita (p. 329). O limite é a legibilidade:

Aqui são discutidas estruturas de controle. São definidas como necessárias somente Sequência (uma sentença após a outra), Seleção (caminho, if) e Iteração (repetir uma ação com base numa condição). As linguagens criam novas estruturas para facilitar a escrita do código. GOTO também foi definido como desnecessário.

Slide 16

Seleção de dois caminhos: o "else pendente"

C: a indentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;  
if (sum == 0)  
    if (count == 0)  
        result = 0;  
else  
    result = 1;  
printf("Resultado: %d", result); // 1
```

Regra de Java, C, C++ e C#
O else pertence ao if mais próximo que ainda não tem else. A indentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x }; // Rust: if é uma expressão (como em F#, p. 334)  
y = x if x > 0 else 2 * x           # Python: expressão condicional (p. 306)
```

Sebesta, LIT ed. - §8.2.1 - p. 330-334 exemplos/07_selecao

Tempo: ~4 min. DEMONSTRAÇÃO: else_pendente.c. Pergunte o valor de result ANTES de rodar.

Questões de projeto (p. 330): qual é a forma e o tipo da expressão de controle? Como são especificadas as cláusulas então e senão? Qual é o significado de seletores aninhados?
Exemplo do livro (p. 331): a indentação sugere que o else pertence ao primeiro if, mas em Java, C, C++ e C# o else casa com o if anterior MAIS PRÓXIMO que ainda não tem else. A indentação é ignorada pelo compilador (exceto em Python e F#). É a ambiguidade da gramática vista no Cap.

O problema do "else pendente" ocorre quando a indentação engana visualmente, mas linguagens como Java e C associam o else ao if mais próximo; outras exigem chaves para evitar essa ambiguidade. Além disso, algumas linguagens tratam o if como uma expressão que retorna um valor.

Slide 17

Seleção múltipla: do switch de C ao match

C - sem break, o controle "cai" (p. 335)

```
switch (index) {
  case 1:
  case 3: odd += 1;
  case 2:
  case 4: even += 1;
  default: printf("erro\n");
}
```

Java 14+ - switch como expressão

```
String tipo = switch (dia) {
  case SAB, DOM -> "fim de semana";
  case SEG, TER, QUA,
  QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust - match exaustivo

```
match n {
  1..=0 => "negativo",
  0 => "zero",
  1..=9 => "um dígito",
  _ => "vários dígitos",
} // sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige fallthrough. C#: cair implicitamente é proibido (p. 336). Python 3.10: match com padrões e guardas.

Tempo: ~5 min. DEMONSTRAÇÃO: switch.c (exemplo do livro), Switch.java, switch.go e selecao.rs.

Questões de projeto (p. 334-335): qual é o tipo da expressão de controle? Como os segmentos são especificados? Só UM segmento executa por vez? Como tratar valores não representados? C (p. 335): o livro chama o switch de "relativamente primitivo". Sem break, o controle CAI nos casos seguintes. No exemplo do livro, com index = 1, executa odd += 1, even += 1 E a mensagem de erro do default. O break é um "goto restrito".

O slide aborda a evolução das estruturas de seleção múltipla, contrastando o switch da linguagem C, onde a execução avança de forma implícita para os casos seguintes se não houver a instrução break. Linguagens mais modernas, como Java, Go e Rust, corrigem esse comportamento proibindo a queda implícita entre casos e adicionando recursos avançados, como a obrigatoriedade de cobrir todas as opções possíveis (exaustividade) utilizando o bloco match.

Slide 18

Laços controlados por contador

C - o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {
  printf("%d ", i);
  i += 2; // altera a contagem
} // 0 3 6 9
```

Python - o for percorre um iterável

```
for i in range(0, 10):
  print(i, end=" ")
  i += 2 # não muda nada
# 0 1 2 3 4 5 6 7 8 9
```

for i in range(10, 0, -3): # 10 7 4 1 ...

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Tempo: ~4 min. DEMONSTRAÇÃO: laço_for.c e laço_for.py, com a mesma ideia (alterar a variável dentro do laço) e resultados diferentes.

Questões de projeto (p. 342): qual é o tipo e o escopo da variável do laço? É permitido alterar a variável ou os parâmetros dentro do corpo, e isso afeta o controle? Os parâmetros são avaliados uma vez ou a cada iteração? C (p. 342-344): o for é a forma mais flexível. As três expressões são opcionais, podem ser múltiplas (com vírgula), são avaliadas a cada iteração, e a variável pode ser modificada no corpo. O livro mostra que ele é, na verdade, um laço lógico disfarçado. No exemplo, i += 2 dentro do corpo faz o

O slide compara como diferentes linguagens implementam laços controlados por contador e as regras sobre alterar a variável de controle. Enquanto em C modificar a variável dentro do corpo afeta diretamente a contagem e as próximas iterações, em Python isso não altera o fluxo, pois o laço apenas consome os valores de um iterável predefinido.

Slide 19

The slide is titled "Laços lógicos, saídas e iteradores". It contains three code snippets: a Java snippet using a labeled loop with a break statement, a JavaScript snippet using a for...in loop, and a C snippet using a while loop with a break statement. A yellow box highlights a warning about the post-test nature of do-while loops. The slide is part of a presentation titled "Aula007_ExpressoesControl" and is slide 19 of 22.

Laços lógicos, saídas e iteradores

Java - break rotulado (p. 348)

```
externo:
for (int i = 0; i < m.length; i++)
for (int j = 0; j < m[i].length; j++)
if (m[i][j] != 0) {
    System.out.println(i + ", " + j);
    break externo; // sai dos DOIS laços
}
```

JavaScript - for...in = for...of

```
const notas = [10, 20, 30];
let total = 0;
for (const i in notas) total += i;
console.log(total); // "0012": chaves!

total = 0;
for (const v of notas) total += v;
console.log(total); // 60: valores
```

Zune [2008] - o laço lógico sem progresso

```
while (dias > 365) {
    if (bissento(ano)) {
        if (dias > 366) { dias -= 366; ano++; }
        // dias == 366: nada muda, laço infinito!
    } else { dias -= 365; ano++; }
}
```

Pós-teste (do-while) executa ao menos uma vez, e isso é fácil de esquecer (p. 347).

Tempo: ~5 min. DEMONSTRAÇÃO: zune.c (termina detectando o laço infinito), Rotulos.java e iteracao.js.

Laços lógicos (8.3.2, p. 345-347): pré-teste (while) ou pós-teste (do-while). O livro avisa que o pós-teste é perigoso, porque o corpo executa ao menos uma vez e os programadores esquecem disso.

Zune (31/12/2008): com dias = 366 num ano bissexto, nenhum ramo altera 'dias' e o while nunca termina. Pergunte: "que invariante o laço deveria garantir?" (a cada volta, 'dias' precisa diminuir).

O slide ilustra os perigos dos laços lógicos, destacando laços infinitos por falta de progresso na condição e a execução não intencional em pós-testes (do-while). Também demonstra o uso prático de saídas rotuladas para interromper múltiplos laços aninhados em Java e as armadilhas de iteração no JavaScript ao confundir o acesso por chaves (for...in) com o acesso por valores (for...of).

Slide 20

The slide is titled "Desvio incondicional: o goto". It contains a C code snippet using goto statements to implement a certificate validation function. It also includes a quote from Dijkstra (1968) and a note about the historical debate on goto. The slide is part of a presentation titled "Aula007_ExpressoesControl" and is slide 20 of 22.

Desvio incondicional: o goto

"goto fail" (Apple, 2014) - versão simplificada

```
int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
    goto fail; // duplicada: desvia SEMPRE
    if ((err = verificaassinatura()) != 0)
        goto fail; // nunca é alcançada
fail:
    return err; // 0 = "certificado válido"
}
```

Dijkstra (1968)
"Go To Statement Considered Harmful"

Quem tem goto hoje
C, C++, C#, Go, Java não tem (p. 353)

Gotos restritos
break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de indentação enganosa), o bug teria sido pegado.

Tempo: ~4 min. DEMONSTRAÇÃO: goto_fail.c. Pergunte o que valida_certificado() devolve.

Sebesta (p. 352): o goto é a sentença de controle mais poderosa, já que todas as outras podem ser construídas com goto e um seletor. Esse poder é justamente o perigo: sem restrições, o goto torna o programa difícil de ler, pouco confiável e caro de manter. A legibilidade é melhor quando a ordem de execução segue a ordem do texto.

Debate histórico: Dijkstra, "Go To Statement Considered Harmful" (1968). Knuth defendeu alguns usos em "Structured Programming with go to

O slide debate o uso do desvio incondicional goto, que é amplamente criticado por comprometer a legibilidade e a segurança do código, perigo este bem exemplificado pela famosa falha de segurança "goto fail" da Apple causada por má indentação e falta de chaves. Apesar de o goto clássico ser evitado ou até inexistente em algumas linguagens, desvios restritos como break, continue e return são amplamente aceitos e utilizados para melhorar a clareza do programa.

Slide 21

The screenshot shows a presentation slide titled "Comandos protegidos de Dijkstra". The slide is part of a presentation named "Aula007_ExpressoesControl" in a PDF viewer. The slide content is as follows:

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x = y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no if) ou fim do laço (no do ... od)
- Base do **CSP** de Hoare (1978) e do select de Go

Go - select, herdeiro dos comandos protegidos

```
a <- "a" // os dois canais
b <- "b" // estão prontos

select { // Go sorteia uma guarda
case x := <- a:
contagem[x]++
case y := <- b:
contagem[y]++
}
```

// 1000 repetições:
// map[a:505 b:495] (muda a cada execução)

Sebasta, 127 ed. [8.5. p. 353-355 - exemplos/02_sebasta_protegido

Tempo: ~4 min. DEMONSTRAÇÃO: rode select.go e guardas.py DUAS vezes. A saída muda, mas o resultado final de guardas.py é sempre ordenado.

Sebasta (p. 353-355): Dijkstra (1975) propôs sentenças de seleção e laço cujo objetivo era apoiar a prova de corretude durante o desenvolvimento. Seleção if ... fi: TODAS as guardas são avaliadas. Se mais de uma é verdadeira, uma delas é escolhida de forma NÃO DETERMINÍSTICA. Se nenhuma é verdadeira, é erro de execução. Isso obriga o programador a pensar em todos os casos.

O slide apresenta os comandos protegidos de Dijkstra, que avaliam todas as condições (guardas) simultaneamente e executam uma escolha não determinística caso mais de uma seja verdadeira. Esse modelo teórico fundamenta mecanismos práticos modernos, como a instrução select da linguagem Go, que seleciona de forma aleatória entre múltiplos canais prontos para execução.

Slide 22

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

1 - JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
  total += p;
console.log("Total: " + total);
```

2 - Python

```
def desconto(preco, d=None):
    d = d or 10 # padrão: 10%
    return preco * (100 - d) / 100
print(desconto(200, 0))
```

Tempo: 10 min (7 para as duplas, 3 para a correção). Pode valer como atividade avaliativa em sala.
Para cada trecho: (a) qual é a saída? (b) qual regra da linguagem causa o problema? (c) como corrigir?

GABARITO:

1) JavaScript: imprime "Total: 0012". O for...in percorre as chaves (strings), e o + concatena. Correção: for...of, ou precos.reduce((a, b) => a + b, 0).
2) Python: imprime 180.0, mas o esperado era 200.0 (desconto zero). O "d or 10" trata o 0 como falso. Correção: "if d is None: d = 10". Em

1. JavaScript

a) Saída: Total: 0012.

b) Regra do problema: O laço for...in itera sobre os índices do vetor (as chaves "0", "1", "2") como strings, e não sobre os valores numéricos. A soma de um número com uma string resulta em concatenação no JavaScript.

c) Correção: Substituir o for...in por for...of, que itera corretamente sobre os valores: for (const p of precos)

2. Python

a) Saída: 180.0

b) Regra do problema: Em Python, o número 0 é considerado um valor falso (falsy). A expressão lógica `d = d or 10` avalia `0 or 10` e devolve 10, ignorando o desconto de 0% que foi passado como argumento.

c) Correção: Verificar explicitamente se a variável é None em vez de usar o operador or. A linha deve ser substituída por: `if d is None: d = 10`.

Slide 23

The screenshot shows a presentation slide titled "Exercício: encontre o bug (2/3)". The slide is part of a presentation named "Aula007_ExpressoesControle.pptx". The slide content includes two code snippets, one in Java and one in C, and a "GABARITO:" (Answer) section at the bottom.

Exercício: encontre o bug (2/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

3 - Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 - C

```
int saldo = 100, saque = 50;
if (saque > saldo)
    printf("saldo insuficiente\n");
printf("fim\n");
```

GABARITO:

3) Java: imprime "inválido". O switch sem break cai do case 2 até o default. Correção: break em cada caso ou switch com -> (Java 14+).

4) C: imprime "saldo insuficiente" e "fim", mesmo com saque < saldo. O ";" logo depois do if é uma sentença vazia, então o printf fica fora do if. Correção: chaves sempre. O gcc com -Wall avisa (-Wempty-body).

3. Java

a) Saída: inválido.

b) Regra que causa o problema: A ausência da instrução break no final de cada caso faz com que o fluxo de execução "caia" implicitamente para os casos seguintes (comportamento conhecido como fall-through). O código entra no case 2, mas continua a executar o case 3 e, em seguida, o default, sobrescrevendo o valor da variável nome com a última atribuição.

c) Como corrigir: Adicionar a instrução break; no final de cada bloco case para forçar a saída da estrutura switch após a atribuição.

4. C

a) Saída: O código imprimirá saldo insuficiente seguido por fim.

b) Regra que causa o problema: A presença do ponto e vírgula (;) imediatamente a seguir à condição do if cria uma instrução nula/vazia. A indentação do código engana o programador, pois a linha printf("saldo insuficiente\n"); encontra-se fora da condição e será sempre executada, independentemente do valor do saldo.

c) Como corrigir: Remover o ponto e vírgula (;) que se encontra a seguir aos parênteses da condição do if. É também boa prática envolver o bloco condicional com chaves { } para evitar ambiguidades futuras.

Slide 24



Exercício: encontre o bug (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

5 - Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 - Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

GABARITO:

5) Go: imprime "dentro: 20" e "fora: 10". O `:=` dentro do if cria uma NOVA variável x (sombreamento) em vez de atribuir à de fora. Correção: `x = x * 2`. Liga com escopo (aula de nomes e vinculações).

6) Java: imprime "0.0%". Divisão inteira `7 / 10 = 0`, feita ANTES do alargamento para double (atribuição de modo misto). Correção: `acertos * 100.0 / total`, ou `(double) acertos / total * 100`.

5. Go

- a) Saída: O programa imprimirá dentro: 20 e, na linha seguinte, fora: 10.
- b) Regra que causa o problema: Ocorre um "sombreamento" (shadowing) da variável. O operador de declaração curta `:=` utilizado dentro do bloco if cria uma nova variável local x que existe apenas nesse escopo, deixando a variável x externa inalterada.
- c) Como corrigir: Utilizar o operador de atribuição simples `=` em vez de `:=` dentro do bloco if (a linha passaria a ser `x = x * 2`), de forma a atualizar a variável já existente.

6. Java

- a) Saída: A saída será 0.0%.
- b) Regra que causa o problema: Divisão inteira. Como as variáveis `acertos` e `total` são do tipo `int`, a operação `7 / 10` descarta a parte fracionária e resulta em 0, e só depois esse valor é convertido para `double`.
- c) Como corrigir: É necessário forçar a conversão (cast) de um dos operandos para um tipo de ponto flutuante antes de efetuar a divisão. A linha pode ser corrigida para `double taxa = (double) acertos / total * 100`; ou multiplicando primeiro por um valor decimal, como `acertos * 100.0 / total`;

Slide 25

Aula007_Expressões Condicionais .PPTX ☆ 🔒 📄

Arquivo Editar Ver Ajuda

🔍 🖨️ 🔍 55% 🔍 🔗 Semente ver

22

Questão em dupla: armadilhas e como as linguagens ajudam

23

Questão em dupla: armadilhas e como as linguagens ajudam

24

Questão em dupla: armadilhas e como as linguagens ajudam

25

Bônus: armadilha com o flag --help

26

Mensagem final: as linguagens mais recentes (Go, Rust, Kotlin, Swift) aprenderam com os erros de C.

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	-2 ++ 2, 1 + 2 = "3"	parênteses; JS obriga (-2) ++ 2
Operandos	a + f() com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; ++
Booleanos	0, "", [] são falsos; a > b > c em C	If só com booleano (Java, Go, Rust); ??
Atribuição	If (x = y)	atribuição não é expressão (Python, Go, Rust)
Seleção	switch sem break; else pendente	-->, match exhaustivo, chaves obrigatórias
Laços	for...in no JS; laço sem progresso	for...of, iteradores, break rotulado
Desvio	goto sem restrição	break, continue, return: gotos restritos

Semente, LIP-HL - Cap. 7+8 p. 200-201

Tempo: ~2 min. Peça à turma que complete a coluna da direita oralmente antes de mostrar.
Mensagem final: as linguagens mais recentes (Go, Rust, Kotlin, Swift) aprenderam com os erros de C. Elas exigem booleanos no if, chaves obrigatórias, conversões explícitas e switch/match sem "cair", e fazem da atribuição uma sentença. Cada restrição remove uma classe inteira de bugs, trocando um pouco de facilidade de escrita por confiabilidade, o trade-off do Cap. 1.

Esse slide resume os tópicos apresentados anteriormente.